

Επίθεση SQL Injection

Εισαγωγή

Η επίθεση SQL Injection είναι μια από τις πιο κοινές επιθέσεις σε web εφαρμογές. Αποτελεί μια τεχνική *έγχυσης κώδικα* που εκμεταλλεύεται τις ευπάθειες στην αλληλεπίδραση μεταξύ **web εφαρμογών** και **διακομιστών βάσεων δεδομένων**. Η ευπάθεια εμφανίζεται όταν τα δεδομένα που εισάγει ο χρήστης **δεν ελέγχονται** σωστά μέσα στις εφαρμογές, πριν να σταλούν στη βάση δεδομένων του backend.

Πολλές εφαρμογές web λαμβάνουν δεδομένα από χρήστες και στη συνέχεια τα χρησιμοποιούν απευθείας για την **κατασκευή ερωτημάτων SQL**, ώστε να λάβουν πληροφορίες από τη βάση δεδομένων. Επίσης, χρησιμοποιούν ερωτήματα SQL για να **αποθηκεύσουν** πληροφορίες στη βάση δεδομένων. Όταν όμως τα ερωτήματα SQL **δεν είναι προσεκτικά** κατασκευασμένα, μπορεί να προκύψουν τρωτά σημεία που να επιτρέψουν επιθέσεις **SQL injection**.

Σε αυτό το εργαστήριο, θα χρησιμοποιήσουμε μια **web εφαρμογή**, η οποία περιλαμβάνει κοινά σφάλματα που γίνονται από πολλούς προγραμματιστές. Ο στόχος της άσκησης είναι να αναδείξει τους τρόπους εκμετάλλευσης των σφαλμάτων αυτών, να επιδείξει τη ζημιά που μπορεί να επιφέρουν, και τέλος να εξετάσει τις τεχνικές που μπορούν να βοηθήσουν στην αποφυγή αυτών των επιθέσεων.

Περιβάλλον εργαστηρίου

Η άσκηση αυτή έχει δοκιμαστεί στην εικονική μηχανή **SEEDUbuntu - Ubuntu 16.04**. Για τις ανάγκες της άσκησης χρησιμοποιείται μια **web εφαρμογή**, οποία έχει αναπτυχθεί για το σκοπό αυτό.

Web Server

Η φιλοξενία της εφαρμογής γίνεται στον **Apache Web Server**, ο οποίος είναι εγκατεστημένος στην εικονική μηχανή και εκτελείται αυτόματα κατά την εκκίνηση. Χρησιμοποιείται η λειτουργία *εικονικής φιλοξενίας* ώστε να μπορεί να φιλοξενήσει πολλούς ιστότοπους (ή διευθύνσεις URL) στο ίδιο μηχάνημα. Οι οδηγίες διαμόρφωσης περιέχονται στο αρχείο `000-default.conf` που βρίσκεται στον κατάλογο `"/etc/apache2/sites-available"`.

Μέσα στο αρχείο διαμόρφωσης, κάθε web εφαρμογή διαθέτει ένα μπλοκ `VirtualHost` που καθορίζει τη διεύθυνση URL για τον ιστότοπο και τον κατάλογο στο σύστημα αρχείων που περιέχει τον κώδικα για τον ιστότοπο., σύμφωνα με την παρακάτω μορφή:

```
<VirtualHost *>
    ServerName http://www.example1.com
    DocumentRoot /var/www/Example_1/
< VirtualHost>

<VirtualHost *>
    ServerName http://www.example2.com
    DocumentRoot /var/www/Example_2/
< VirtualHost>
```

Ο κατάλογος στον οποίο είναι εγκατεστημένη η συγκεκριμένη εφαρμογή και η διεύθυνση URL για την πρόσβαση σε αυτήν (μέσω του browser της εικονικής μηχανής) είναι οι εξής:

```
URL: http://www.SEEDLabSQLInjection.com
Directory: /var/www/SQLInjection
```

Η παραπάνω διεύθυνση URL είναι **διαθέσιμη μόνο μέσα από την εικονική μηχανή**, επειδή έχει τροποποιηθεί το αρχείο **/etc/hosts** ώστε να αντιστοιχιστεί το *domain name* κάθε διεύθυνσης URL στην τοπική διεύθυνση IP του εικονικού μηχανήματος (**127.0.0.1**).

Έχοντας πρόσβαση στον παραπάνω κατάλογοι μπορούμε να κάνουμε τροποποιήσεις στον κώδικα της εφαρμογής. Εάν γίνει μια οποιαδήποτε **αλλαγή**, είτε στη **διαμόρφωση** ή στον **κώδικα** κάποιας εφαρμογής, θα πρέπει να γίνει επανεκκίνηση του *Apache Server*, ώστε η αλλαγή αυτή να εφαρμοστεί. Οι ακόλουθες εντολές χρησιμοποιούνται για να σταματήσουν και να εκκινήσουν τον Apache Server αντίστοιχα:

```
$ sudo service apache2 stop
$ sudo service apache2 start

ή

$ sudo service apache2 restart
```

Database Server

Η **βάση δεδομένων** της εφαρμογής φιλοξενείται σε έναν **MySQL Server**, που είναι ήδη εγκατεστημένος και εκτελείται αυτόματα κατά την εκκίνηση της εικονικής μηχανής.

Το όνομα του χρήστη-διαχειριστή της ΒΔ είναι **root** και ο κωδικός πρόσβασης είναι **seedubuntu**. Η σύνδεση στην κονσόλα του **MySQL Server** μπορεί να γίνει χρησιμοποιώντας την ακόλουθη εντολή:

```
$ mysql -u root -pseedubuntu
```

Εφαρμογή Web

Η web εφαρμογή που χρησιμοποιούμε για την επίδειξη των δραστηριοτήτων της άσκησης, είναι μια απλή εφαρμογή **διαχείρισης υπαλλήλων** η οποία φιλοξενείται στη διεύθυνση **www.SEEDLabSQLInjection.com**. Οι εργαζόμενοι μπορούν μέσω αυτής να **δούνε** και να **ενημερώσουν** τα προσωπικά τους στοιχεία σε μια **βάση δεδομένων**. Η εφαρμογή υποστηρίζει δύο βασικές κατηγορίες χρηστών:

- Ο **διαχειριστής (Admin)**, είναι ένας χρήστης με προνόμια διαχείρισης, ο οποίος μπορεί να διαχειριστεί τις πληροφορίες προφίλ όλων των εργαζομένων.
- Ο **υπάλληλος**, είναι ένας κανονικός χρήστης, ο οποίος μπορεί να δει ή να ενημερώσει μόνο κάποιες από τις δικές του πληροφορίες προφίλ.

Όλες οι αρχικές πληροφορίες που έχουν ήδη εισαχθεί στη βάση δεδομένων περιγράφονται στον παρακάτω πίνακα:

Name	Employee ID	Password	Salary	Birthday	SSN	Nickname	Email	Address	Phone#
Admin	99999	seedadmin	400000	3/5	43254314				
Alice	10000	seedalice	20000	9/20	10211002				
Boby	20000	seedboby	50000	4/20	10213352				
Ryan	30000	seedryan	90000	4/10	32193525				
Samy	40000	seedsamy	40000	1/11	32111111				
Ted	50000	seedted	110000	11/3	24343244				

Δραστηριότητα 1: Εξοικείωση με τις εντολές SQL

Ο στόχος αυτής της δραστηριότητας είναι η εξοικείωση με τις εντολές SQL χρησιμοποιώντας την κονσόλα του MySQL Server. Η βάση δεδομένων που χρησιμοποιούμε ονομάζεται **Users** και περιέχει έναν πίνακα που ονομάζεται **credential**. Ο πίνακας αποθηκεύει τις προσωπικές πληροφορίες (π.χ. *eid*, *password*, *salary*, *ssn* κ.λπ.) κάθε υπαλλήλου.

Η φόρτωση της βάσης δεδομένων **Users** στην κονσόλα του MySQL Server γίνεται χρησιμοποιώντας την ακόλουθη εντολή:

```
mysql> use Users;
```

Για να δούμε στην κονσόλα ποιοι πίνακες υπάρχουν στη βάση δεδομένων **Users**, μπορείτε να χρησιμοποιήσετε την ακόλουθη εντολή, η οποία θα τυπώσει στην έξοδο όλους τους πίνακες της επιλεγμένης βάσης δεδομένων.

```
mysql> show tables;
```

Σκοπός της **Δραστηριότητας 1** είναι, αφού συνδεθείτε με τη βάση **Users**, να εκτελέσετε τα κατάλληλα **ερωτήματα SQL** ώστε να πραγματοποιήσετε τις παρακάτω ενέργειες:

- **Δραστηριότητα 1.1: Εκτύπωση πληροφοριών πίνακα**

Να τυπώσετε (στο terminal) πληροφορίες για τη δομή του πίνακα **credential** (δηλαδή τα πεδία του και τους αντίστοιχους τύπους).

- **Δραστηριότητα 1.2: Εκτύπωση πληροφοριών χρηστών**

Να τυπώσετε (στο terminal) όλα τα περιεχόμενα του πίνακα **credential**. Στη συνέχεια να τυπώσετε όλες τις πληροφορίες μόνο του υπαλλήλου με το όνομα **Samy**.

- **Δραστηριότητα 1.3: Εισαγωγή νέων χρηστών**

Να εισάγετε δυο νέους χρήστες (με όλα τους τα πεδία συμπληρωμένα **εκτός από το password**), οι οποίοι θα χρησιμοποιηθούν για τις επιθέσεις. Ο ένας από τους νέους χρήστες θα πρέπει να έχει **name** το μικρό σας όνομα.

- **Δραστηριότητα 1.4: Ενημέρωση password των νέων χρηστών**

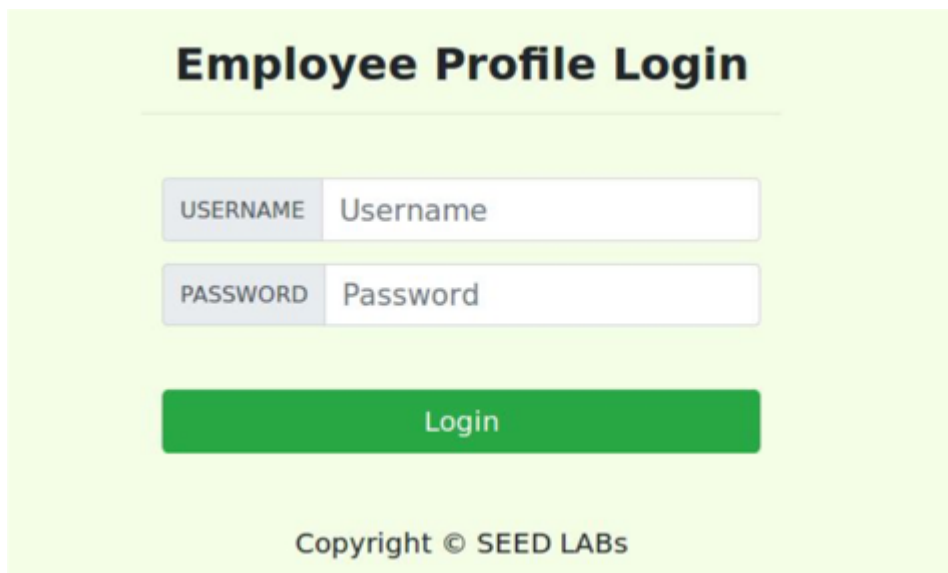
Στη δραστηριότητα αυτή καλείστε να επιλέξετε **password** για τους δυο νέους χρήστες και να ενημερώσετε τις εγγραφές τους στη ΒΔ. Για λόγους ασφαλείας, το πεδίο **password** δεν αποθηκεύεται σε αναγνώσιμη (plain) μορφή. Αυτό που αποθηκεύεται είναι η **σύνοψη**, η οποία παράγεται από έναν αλγόριθμο **hash**. Αυτό σημαίνει ότι, πριν ενημερώσετε τις εγγραφές, θα πρέπει να βρείτε τις τιμές **hash** των επιλεγμένων **passwords**. Ο αλγόριθμος που χρησιμοποιείται στη συγκεκριμένη εφαρμογή είναι ο **SHA1**.

Στην αναφορά σας, εκτός από την επεξήγηση των ενεργειών σας, θα πρέπει να συμπεριλάβετε στιγμιότυπα οθόνης με τις εντολές που χρησιμοποιήσατε και τα αποτελέσματα αυτών. Μετά από κάθε ενέργεια εισαγωγής ή ενημέρωσης θα πρέπει να εκτελείτε το κατάλληλο **SELECT query** που θα δείχνει την επιτυχία της ενέργειας.

Δραστηριότητα 2: Επίθεση SQL Injection σε εντολές SELECT

Η **SQL Injection** είναι βασικά μια τεχνική μέσω της οποίας οι επιτιθέμενοι μπορούν να εκτελέσουν τις δικές τους (κακόβουλες) εντολές **SQL**. Μέσω των κακόβουλων εντολών **SQL**, οι εισβολείς μπορούν να **υποκλέψουν** πληροφορίες για τα θύματα από τη βάση δεδομένων ή ακόμα χειρότερα, μπορεί **να κάνουν αλλαγές** στη βάση δεδομένων. Η συγκεκριμένη εφαρμογή διαχείρισης υπαλλήλων περιέχει κοινά σφάλματα που κάνουν οι προγραμματιστές.

Στη δραστηριότητα 2 θα χρησιμοποιήσουμε τη σελίδα σύνδεσης χρηστών (login page) της εφαρμογής **www.SEEDLabSQLInjection.com**. Η σελίδα σύνδεσης (Εικόνα 1) ζητάει από τους χρήστες να εισάγουν το **όνομα χρήστη** και τον **κωδικό πρόσβασης**. Η εφαρμογή επικυρώνει τον χρήστη με βάση αυτά τα δύο στοιχεία, επομένως μόνο οι υπάλληλοι που γνωρίζουν τους κωδικούς τους επιτρέπεται να συνδεθούν. Η αποστολή σας, ως εισβολέας, είναι **να συνδεθείτε** στην web εφαρμογή **ΧΩΡΙΣ** να γνωρίζετε τα διαπιστευτήρια κάποιου υπαλλήλου.



Εικόνα 1. Login page

Για να βοηθηθείτε σχετικά με αυτή την δραστηριότητα, παρουσιάζεται αναλυτικά πώς εφαρμόζεται ο έλεγχος ταυτότητας στην εφαρμογή (σε επίπεδο κώδικα):

Ο κώδικας **PHP** που χρησιμοποιείται για τη διεξαγωγή επαλήθευσης χρήστη βρίσκεται στο αρχείο **unsafe_home.php**, το οποίο είναι αποθηκευμένο στον κατάλογο **/var/www/SQLInjection**. Το ακόλουθο απόσπασμα κώδικα δείχνει τον τρόπο επαλήθευσης των χρηστών.

```

$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);
...
$sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
        nickname, Password
        FROM credential
        WHERE name= '$input_uname' and Password='$hashed_pwd'";
$result = $conn->query($sql);

// The following is Pseudo Code
if(id != NULL) {
    if(name=='admin') {
        return All employees information;
    } else if (name !=NULL){
        return employee information;
    }
} else {
    Authentication Fails;
}

```

Η παραπάνω εντολή SQL επιλέγει τις προσωπικές πληροφορίες των εργαζομένων, όπως το *id*, *name*, *salary*, *ssn* κλπ., από τον πίνακα **credential**. Η εντολή SQL χρησιμοποιεί δύο μεταβλητές **input_uname** and **hashed_pwd**, όπου η **input_uname** περιέχει τη συμβολοσειρά που πληκτρολογούν οι χρήστες στο πεδίο **ονόματος χρήστη** της σελίδας σύνδεσης, ενώ η **hashed_pwd** περιέχει την hashed τιμή (με τον αλγόριθμο sha1) του **κωδικού πρόσβασης** που πληκτρολογεί ο χρήστης. Το πρόγραμμα ελέγχει αν οποιαδήποτε εγγραφή ταιριάζει με το παρεχόμενο **όνομα χρήστη** και **κωδικό πρόσβασης**. Αν υπάρχει αντιστοιχία, ο χρήστης έχει πιστοποιηθεί με επιτυχία και αποκτά πρόσβαση στις αντίστοιχες πληροφορίες. Εάν δεν υπάρχει αντιστοιχία, ο έλεγχος ταυτότητας αποτυγχάνει.

Έχοντας υπόψη τα παραπάνω, θα πρέπει να ολοκληρώσετε τις παρακάτω δραστηριότητες:

- **Δραστηριότητα 2.1: Επίθεση SQL Injection στη σελίδα login.**

Σκοπός της **ΔΡΑΣΤΗΡΙΟΤΗΤΑΣ 2.1** είναι να **συνδεθείτε** στη web εφαρμογή ως **διαχειριστής** από τη **σελίδα σύνδεσης** (login page), ώστε να μπορείτε να δείτε τις πληροφορίες όλων των εργαζομένων. Υποθέτουμε ότι γνωρίζετε μόνο το όνομα λογαριασμού του διαχειριστή (**admin**), αλλά **ΔΕΝ** έχετε τον κωδικό πρόσβασης. Πρέπει να αποφασίσετε τι να πληκτρολογήσετε στα πεδία **Username** και **Password** για να επιτύχει η επίθεση.

Στην αναφορά σας θα πρέπει να συμπεριλάβετε στιγμιότυπα οθόνης με το input που πληκτρολογήσατε στα πεδία και το αποτέλεσμα της επίθεσης. Επίσης, να εξηγήσετε γιατί πέτυχε η επίθεση, με βάση το παραπάνω κομμάτι κώδικα **php** που περιγράφει την διαδικασία **login**.

- **Δραστηριότητα 2.2: Επίθεση SQL Injection από τη γραμμή εντολών.**

Σκοπός της **Δραστηριότητα 2.2** είναι να επαναλάβετε τη **δραστηριότητα 2.1**, αλλά **ΧΩΡΙΣ** να χρησιμοποιήσετε την **login page**. Αντί αυτής, μπορείτε να χρησιμοποιήσετε εργαλεία **γραμμής εντολών** τα οποία μπορούν να στέλνουν αιτήσεις **HTTP**, όπως είναι το πρόγραμμα **curl**. Θα πρέπει να προσέξετε ότι, αν θέλετε να συμπεριλάβετε **πολλαπλές παραμέτρους** σε αιτήματα **HTTP**, πρέπει να βάλετε τη διεύθυνση **URL** και τις παραμέτρους μέσα σε ένα ζεύγος **μονών εισαγωγικών** ('). Διαφορετικά, οι ειδικοί χαρακτήρες που χρησιμοποιούνται για τον διαχωρισμό παραμέτρων (όπως το **&**) θα ερμηνευτούν από το πρόγραμμα κελύφους, αλλάζοντας την έννοια της εντολής. Το παρακάτω **παράδειγμα** δείχνει τον τρόπο αποστολής ενός αιτήματος **GET HTTP request** στην εφαρμογή μας, με δύο παραμέτρους (όνομα χρήστη και κωδικό πρόσβασης) που επισυνάπτονται:

```
$ curl 'www.SEEDLabSQLInjection.com/unsafe_home.php?username=alice&password=111'
```

Επίσης, εάν πρέπει να συμπεριλάβετε **ειδικούς χαρακτήρες** στα πεδία **ονόματος χρήστη (username)** ή **κωδικού πρόσβασης (password)**, πρέπει να τα κωδικοποιήσετε σωστά (**hex ASCII**) αλλιώς θα αλλάξει η έννοια των αιτημάτων σας. Για παράδειγμα, εάν θέλετε να συμπεριλάβετε ένα μονό εισαγωγικό (') σε αυτά τα πεδία, θα πρέπει αντί γι' αυτό να χρησιμοποιήσετε τον συμβολισμό **%27**. Εάν θέλετε να συμπεριλάβετε κενό (whitespace), θα πρέπει να χρησιμοποιήσετε τον συμβολισμό **%20**.

*Σε αυτή την δραστηριότητα, θα πρέπει να χειριστείτε την κωδικοποίηση **HTTP** κατά την αποστολή αιτημάτων χρησιμοποιώντας το **curl**. Στην αναφορά σας θα πρέπει να συμπεριλάβετε στιγμιότυπα οθόνης με την εντολή που πληκτρολογήσατε και το αποτέλεσμα της επίθεσης.*

Για να χειριστείτε καλύτερα το περιεχόμενο **HTML που λαμβάνετε ως απάντηση στο **terminal**, μπορείτε να χρησιμοποιήσετε έναν **text-web browser** όπως είναι ο **Lynx**¹. Το εργαλείο αυτό μπορεί να εγκατασταθεί στην εικονική μηχανή χρησιμοποιώντας την εντολή:*

```
$ sudo apt install lynx
```

Αφού το εγκαταστήσετε, μπορείτε να το χρησιμοποιήσετε μεταβάλλοντας κατάλληλα την εντολή του παραπάνω παραδείγματος ως εξής:

```
$ curl 'www.SEEDLabSQLInjection.com/unsafe_home.php?....' | lynx --stdin
```

¹ <https://lynx.invisible-island.net>

- **Δραστηριότητα 2.3: Εκτέλεση πολλαπλών εντολών SQL.**

Στις δύο παραπάνω επιθέσεις, μπορούμε μόνο να **παρακάμψουμε** μηχανισμούς ελέγχου ή να **υποκλέψουμε** πληροφορίες από τη βάση δεδομένων. Θα ήταν ακόμα πιο επικίνδυνο εάν μπορούσαμε να **τροποποιήσουμε τη βάση δεδομένων** χρησιμοποιώντας την ίδια ευπάθεια στη σελίδα σύνδεσης. Ένα τρόπος να συμβεί αυτό είναι να χρησιμοποιήσετε την επίθεση **SQL Injection** για να μετατρέψετε **μια** εντολή **SQL** σε **δύο**, με τη δεύτερη να είναι η δήλωση **ενημέρωσης** (update statement) ή **διαγραφής** (delete statement). Στην SQL, το **semicolon (;)** χρησιμοποιείται για να διαχωρίσει δύο εντολές **SQL**.

Σκοπός της **Δραστηριότητας 2.3** είναι να χρησιμοποιήσετε τα πεδία **username** ή/και **password** της σελίδας σύνδεσης (login page) για να **διαγράψετε** μια εγγραφή ή να **ενημερώστε** ένα οποιοδήποτε πεδίο στη βάση δεδομένων. Επίσης, επιχειρήστε – πάλι μέσω εκτέλεσης πολλαπλών εντολών SQL από τη σελίδα σύνδεσης - να ανακτήσετε το συνθηματικό κάποιου χρήστη (ουσιαστικά, το αποτύπωμα (hash value) αυτού).

*Στην αναφορά σας, περιγράψτε τον τρόπο με τον οποίο μπορείτε να χρησιμοποιήσετε τη σελίδα σύνδεσης για να εκτελέσετε πολλαπλές εντολές **SQL**. Δοκιμάστε την επίθεση και καταγράψτε τα αποτελέσματα. Εξηγήστε την επιτυχία ή την αποτυχία της επίθεσης, λαμβάνοντας υπόψη το κομμάτι κώδικα *php* που περιγράφει την διαδικασία **Login**.*

- **Δραστηριότητα 2.4: Ανάκτηση συνθηματικού (επίθεση τύπου rainbow)**

Από τη δραστηριότητα 2.3, είτε θα έχετε καταφέρει να ανακτήσετε το συνθηματικό κάποιου χρήστη (αν η επίθεση είναι επιτυχής) είτε όχι (αν η επίθεση δεν μπορεί να πραγματοποιηθεί). Σε κάθε περίπτωση, ανεξαρτήτως της έκβασης της δραστηριότητας 2.3, θεωρείστε ότι η επίθεση είναι επιτυχής και ότι μπορείτε να ανακτήσετε τα **αποτυπώματα** (hash values) των συνθηματικών όλων των χρηστών (τα οποία μπορείτε ούτως ή άλλως να τα έχετε, για τους σκοπούς αυτής της Δραστηριότητας, από τη Δραστηριότητα 1.2). Θεωρητικά, από τα αποτυπώματα, δεν είναι εφικτή η ανάκτηση των ίδιων των συνθηματικών αφού οι hash functions είναι μη αναστρέψιμες. Είναι όμως πάντα έτσι;

Ένας επιτιθέμενος μπορεί να προ-υπολογίσει τα αποτυπώματα λέξεων/συμβολοσειρών οι οποίες είναι πιθανό να επιλεγούν από κάποιον χρήστη, κατασκευάζοντας έτσι ένα υπερ-αρχείο που αποτελείται από πιθανά συνθηματικά και, για κάθε ένα εξ αυτών, το αποτύπωμά του (για διάφορες hash functions). Με αυτόν τον τρόπο, μπορεί να ελέγξει αν ένα αποτύπωμα συνθηματικού υπάρχει σε αυτό το υπερ-αρχείο: σε καταφατική περίπτωση, αυτό σημαίνει ότι αναγνωρίζει απευθείας το συνθηματικό. Πρόκειται για επίθεση τύπου **"rainbow"**.

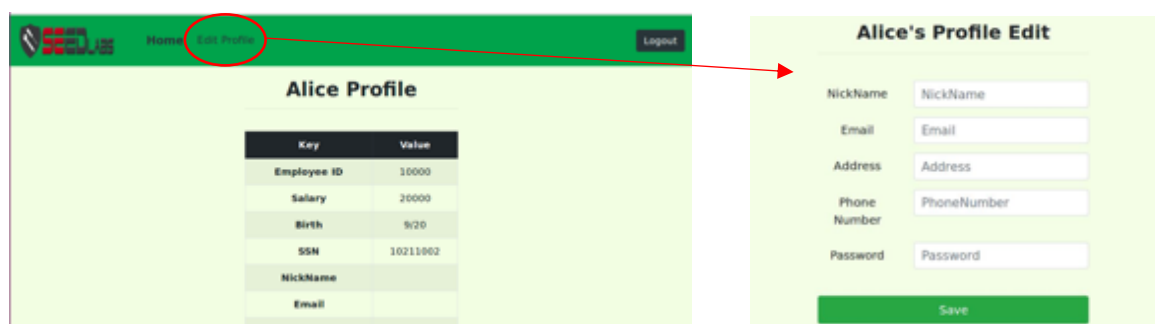
Χρησιμοποιώντας τα αποτυπώματα (hash values) των συνθηματικών που έχετε ανακτήσει (συμπεριλαμβανομένων των συνθηματικών που δημιουργήσατε εσείς), ελέγξτε αν η επίθεση τύπου **"rainbow"** μπορεί να τα αποκαλύψει πλήρως. Για το σκοπό

αυτό μπορείτε να αξιοποιήσετε κατάλληλα online εργαλεία, όπως η ιστοσελίδα <https://hashes.com>, <https://crackstation.net/>.

Στην αναφορά σας, περιγράψτε (χρησιμοποιώντας κατάλληλα screenshots) τον τρόπο με τον οποίο εκτελέσατε την επίθεση Rainbow στα (hashed) συνθηματικά που έχετε ανακτήσει. Σχολιάστε τα αποτελέσματα που λάβατε.

Δραστηριότητα 3: Επίθεση SQL Injection σε δηλώσεις UPDATE

Εάν μια ευπάθεια **SQL Injection** εμφανιστεί σε μια δήλωση **UPDATE**, η πιθανή ζημιά θα είναι πιο σοβαρή, επειδή οι εισβολείς μπορούν να χρησιμοποιήσουν την ευπάθεια για να **τροποποιήσουν** μια βάση δεδομένων. Στην εφαρμογή **Διαχείρισης Υπαλλήλων** υπάρχει μια σελίδα για την **επεξεργασία** του προφίλ χρηστών (Εικόνα 2), που επιτρέπει στους υπαλλήλους να ενημερώνουν τις πληροφορίες του προφίλ τους, συμπεριλαμβανομένου του *nickname*, *email*, *address*, *phone number* και *password*. Για μετάβαση σε αυτή τη σελίδα, οι υπάλληλοι πρέπει πρώτα να συνδεθούν (login) και έπειτα να επιλέξουν **Edit Profile**.



Εικόνα 2. Σελίδα επεξεργασίας προφίλ

Όταν ένας χρήστης αποθηκεύσει (**Save**) τις πληροφορίες του μέσω της σελίδας **Edit Profile**, θα εκτελεστεί το ακόλουθο ερώτημα **SQL UPDATE**:

```
$hashed_pwd = sha1($input_pwd);  
$sql = "UPDATE credential SET  
    nickname=' $input_nickname',  
    email=' $input_email',  
    address=' $input_address',  
    Password=' $hashed_pwd',  
    PhoneNumber=' $input_phonenumber'  
    WHERE ID=$id;";  
$conn->query($sql);
```

Ο κώδικας **PHP** που χρησιμοποιείται για την ενημέρωση των πληροφοριών του προφίλ του υπαλλήλου βρίσκεται στο αρχείο **unsafe_edit_backend.php**. Το αρχείο **PHP** βρίσκεται επίσης στον κατάλογο **/var/www/SQLInjection**.

Έχοντας υπόψη τα παραπάνω, θα πρέπει να ολοκληρώσετε τις παρακάτω δραστηριότητες:

- **Δραστηριότητα 3.1: Τροποποίηση των δικών σας στοιχείων.**

Όπως φαίνεται στη σελίδα **"Edit Profile"**, οι υπάλληλοι μπορούν να ενημερώσουν μόνο το *nickname*, *email*, *address*, *phone number* και *password*. Δεν τους επιτρέπεται να αλλάξουν άλλα στοιχεία, όπως π.χ. το **μισθό** τους (*salary*). Ας υποθέσουμε ότι **εσείς** είστε ένας δυσανεσθημένος υπάλληλος διότι ο προϊστάμενός σας (**Boby**) δεν αύξησε το μισθό σας φέτος.

Σκοπός της **Δραστηριότητας 3.1** είναι, αφού συνδεθείτε με το **λογαριασμό σας** (που δημιουργήσατε στη δραστηριότητα **1.3-1.4**), να αυξήσετε **ΜΟΝΟ** τον δικό σας **μισθό** σε ό,τι ποσό επιθυμείτε, αξιοποιώντας την ευπάθεια SQL Injection στη σελίδα *Edit Profile*. Θα πρέπει να προσέξετε ώστε το query που θα εκτελεστεί **να μην επηρεάσει** τους μισθούς των άλλων υπαλλήλων.

Στην αναφορά σας θα πρέπει να εξηγήσετε τον τρόπο με τον οποίο θα διεξάγετε την επίθεση και να συμπεριλάβετε στιγμιότυπα οθόνης με το **input** που πληκτρολογήσατε στα αντίστοιχα πεδία. Επίσης, θα πρέπει να συμπεριλάβετε screenshots από το **profile page** σας πριν και μετά την επίθεση, ώστε να αποδειχθεί η επιτυχία της.

- **Δραστηριότητα 3.2: Τροποποίηση στοιχείων άλλων χρηστών.**

Αφού αυξήσετε το δικό σας μισθό, αποφασίζετε να βοηθήσετε έναν ακόμα **συναδέλφο** (τον **δεύτερο χρήστη** που δημιουργήσατε στη δραστηριότητα **1.3-1.4**) και κατόπιν να τιμωρήσετε τον προϊστάμενό σας **Boby**.

Σκοπός της **Δραστηριότητας 3.2** είναι, αφού συνδεθείτε με το **λογαριασμό σας**, στη σελίδα *Edit Profile*, να κάνετε τις εξής ενέργειες: (1) να **αυξήσετε** τον μισθό του συναδέλφου σας σε ό,τι ποσό επιθυμείτε, (2) να **μειώσετε** τον μισθό του χρήστη **Boby** σε **\$1**, (3) να αλλάξετε το **τηλέφωνο** του χρήστη **Boby**. Οι επιθέσεις δεν είναι απαραίτητο να γίνουν σε μια ενέργεια, αλλά σε όσες χρειαστούν. Όπως και προηγουμένως, θα πρέπει να προσέξετε ώστε τα queries που θα εκτελεστούν **να μην επηρεάσουν** τα στοιχεία των άλλων υπαλλήλων.

Στην αναφορά σας θα πρέπει να εξηγήσετε τον τρόπο με τον οποίο θα διεξάγετε την επίθεση και να συμπεριλάβετε στιγμιότυπα οθόνης με το **input** που πληκτρολογήσατε στα αντίστοιχα πεδία. Επίσης, θα πρέπει να συμπεριλάβετε screenshots από τα **profile pages** του **συναδέλφου** σας και του **Boby** πριν και μετά την επίθεση, ώστε να αποδειχθεί η επιτυχία της.

- **Δραστηριότητα 3.3: Τροποποίηση του password άλλων χρηστών.**

Αφού αλλάξετε τον μισθό του **Boby**, εξακολουθείτε να είστε δυσαρεστημένοι, οπότε θέλετε να του αλλάξετε τον κωδικό πρόσβασης και στη συνέχεια να μπορείτε να συνδεθείτε **στο λογαριασμό του** και να προκαλέσετε περαιτέρω ζημιές.

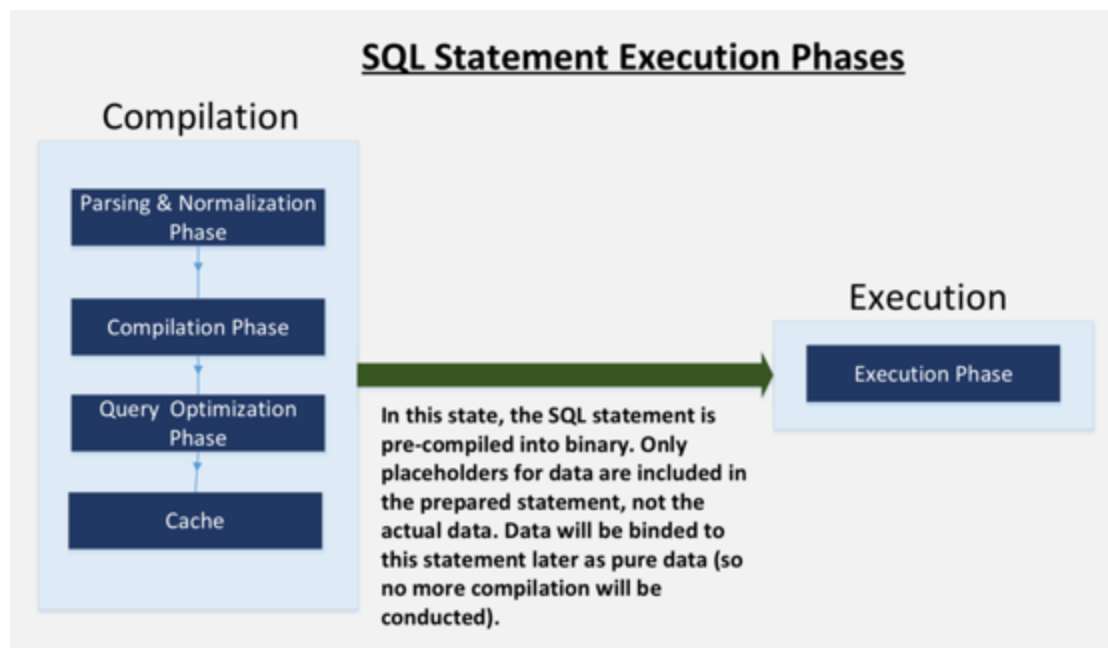
Σκοπός της **Δραστηριότητας 3.3** είναι, αφού συνδεθείτε με το **λογαριασμό σας**, στη σελίδα *Edit Profile*, να **αλλάξετε** το **password ΜΟΝΟ** του χρήστη **Boby** σε κάτι που εσείς γνωρίζετε. Όπως και προηγουμένως, θα πρέπει να προσέξετε ώστε το query που θα εκτελεστεί **να μην επηρεάσει** τα passwords των άλλων υπαλλήλων.

Στην αναφορά σας περιγράψτε πώς μπορείτε να το επιτύχετε και να συμπεριλάβετε στιγμιότυπα οθόνης με το **input** που πληκτρολογήσατε στα αντίστοιχα πεδία. Θα πρέπει επίσης να αποδείξετε ότι μπορείτε να συνδεθείτε με επιτυχία στο λογαριασμό του **Boby** χρησιμοποιώντας το νέο κωδικό πρόσβασης.

Δραστηριότητα 4: Αντίμετρα - Προετοιμασμένη δήλωση

Το βασικό αίτιο που προκαλεί την ευπάθεια **SQL Injection** είναι η αδυναμία της εφαρμογής να διαχωρίσει τον **κώδικα (εντολές)** από τα **δεδομένα**. Κατά την κατασκευή μιας εντολής **SQL**, το πρόγραμμα (π.χ. ο κώδικας **PHP**) γνωρίζει τι αποτελεί δεδομένα και τι αποτελεί εντολές. Όμως, όταν η εντολή **SQL** αποστέλλεται στη βάση δεδομένων, ο διαχωρισμός μεταξύ δεδομένων και κώδικα παύει να υφίσταται. Αυτό έχει σαν αποτέλεσμα οι εντολές που βλέπει ο **διερμηνευτής SQL** να διαφέρουν από τις εντολές που ορίστηκαν από τους προγραμματιστές. Για να επιλυθεί αυτό το πρόβλημα, ο πιο ασφαλής τρόπος είναι να χρησιμοποιείται η **προετοιμασμένη δήλωση** (Prepared Statement) κατά την ανάπτυξη των προγραμμάτων.

Για να κατανοήσουμε πως μια προετοιμασμένη δήλωση εμποδίζει το **SQL Injection**, πρέπει να καταλάβουμε τι συμβαίνει όταν ο database server λαμβάνει ένα query (υπό τη μορφή ενός **SQL Statement**). Η **ροή εργασιών** για την εκτέλεση ενός ερωτήματος παρουσιάζεται στην **Εικόνα 3**.



Εικόνα 3.Ροή εργασιών ενός SQL Statement

Στο ευρύτερο στάδιο της **μεταγλώττισης**, τα ερωτήματα πρώτα περνούν από επιμέρους φάσεις:

- Φάση **ανάλυσης και κανονικοποίησης**, όπου ελέγχεται ένα ερώτημα σε σχέση με τη σύνταξη και τη σημασιολογία.
- Φάση **μεταγλώττισης**, όπου οι λέξεις-κλειδιά (π.χ. **SELECT**, **FROM**, **UPDATE** κ.λπ.) μετατρέπονται σε γλώσσα μηχανής.
- Φάση **βελτιστοποίησης** του ερωτήματος, όπου υπολογίζεται ένας αριθμός διαφορετικών σχεδίων για την εκτέλεση του ερωτήματος, από τα οποία επιλέγεται τελικά το βέλτιστο σχέδιο.

- Φάση προσωρινής αποθήκευσης, όπου όλα σχέδια που τελικά επιλέγονται, αποθηκεύονται στην κρυφή μνήμη (*cache*).

Κάθε φορά που φτάνει ένα νέο ερώτημα, ελέγχεται πρώτα το περιεχόμενο της κρυφής μνήμης. Εάν το ερώτημα υπάρχει ήδη μεταγλωττισμένο στην κρυφή μνήμη, οι φάσεις *ανάλυσης, μεταγλώττισης και βελτιστοποίησης* θα παραλειφθούν. Τέλος, το μεταγλωττισμένο ερώτημα διαβιβάζεται στο **στάδιο εκτέλεσης** όπου ουσιαστικά εκτελείται και επιστρέφει τα αποτελέσματα.

Η προετοιμασμένη δήλωση (**prepared statement**) υπεισέρχεται σε αυτή τη ροή ενεργειών στο στάδιο της μεταγλώττισης. Μια προετοιμασμένη δήλωση θα περάσει από όλες τις φάσεις της μεταγλώττισης και θα μετατραπεί σε ένα προκατασκευασμένο ερώτημα **με κενές θέσεις** για τα δεδομένα.

Για να εκτελεστεί αυτό το προκατασκευασμένο ερώτημα, πρέπει κάποια στιγμή να προστεθούν τα δεδομένα (**bind**), αλλά αυτά τα δεδομένα **δεν θα περάσουν** από το βήμα της σύνταξης. Αντίθετα, συνδέονται **απευθείας** στο προκατασκευασμένο ερώτημα και αποστέλλονται στη μηχανή εκτέλεσης. Επομένως, ακόμα και αν υπάρχει κώδικας SQL μέσα στα δεδομένα, εφόσον αυτά δεν θα περάσουν το βήμα της μεταγλώττισης, ο server θα τα αντιμετωπίσει απλώς **ως μέρος των δεδομένων**, χωρίς ιδιαίτερη σημασία. Αυτός είναι ο τρόπος που η προετοιμασμένη δήλωση αποτρέπει τις επιθέσεις **SQL Injection**.

Ακολουθεί ένα παράδειγμα πώς να χρησιμοποιήσουμε μια προετοιμασμένη δήλωση στην **PHP**. Στην αρχική του μορφή, ο κώδικας που ετοιμάζει το SELECT statement είναι ευάλωτος σε επιθέσεις SQL Injection:

```
$sql = "SELECT name, local, gender
        FROM USER_TABLE
        WHERE id = $id AND password = '$pwd' ";
$result = $conn->query($sql)
```

Χρησιμοποιώντας τον μηχανισμό *προετοιμασμένης δήλωσης*, ο ίδιος κώδικας μπορεί να ξαναγραφεί όπως φαίνεται παρακάτω

```
$stmt = $conn->prepare("SELECT name, local, gender
                        FROM USER_TABLE
                        WHERE id = ? and password = ? ");
// Bind parameters to the query
$stmt->bind_param("is", $id, $pwd);
$stmt->execute();
$stmt->bind_result($bind_name, $bind_local, $bind_gender);
$stmt->fetch();
```

Ουσιαστικά, διαιρούμε τη διαδικασία αποστολής του SQL Statement προς τον database server **σε δύο βήματα**:

- Το πρώτο βήμα είναι να αποστείλουμε **μόνο το τμήμα κώδικα**, δηλ. ένα SQL Statement **χωρίς** τα πραγματικά δεδομένα. Αυτό είναι το βήμα προετοιμασίας. Όπως μπορούμε να δούμε από το παραπάνω απόσπασμα κώδικα, τα πραγματικά δεδομένα αντικαθίστανται από ερωτηματικά (?).

- Σε επόμενη φάση, αποστέλλουμε **τα δεδομένα** χρησιμοποιώντας την μέθοδο **bind_param()**. Ο database server θα αντιμετωπίσει όλα τα στοιχεία που αποστέλλονται σε αυτό το βήμα **μόνο ως δεδομένα** και όχι ως κώδικα και θα τα αντιστοιχίσει στα ερωτηματικά (?) της προετοιμασμένης δήλωσης. Στη μέθοδο **bind_param()**, το πρώτο όρισμα "is" υποδεικνύει τους τύπους των παραμέτρων: "i" σημαίνει ότι τα δεδομένα στη μεταβλητή \$id έχουν τύπο *integer* ενώ "s" σημαίνει ότι τα δεδομένα στη μεταβλητή \$pwd έχουν τύπο *string*.

Έχοντας υπόψη τα παραπάνω, θα πρέπει να ολοκληρώσετε την παρακάτω δραστηριότητα:

Σκοπός της **Δραστηριότητας 4** είναι, χρησιμοποιώντας τον μηχανισμό προετοιμασμένης δήλωσης, να **διορθώσετε** στον κώδικα της εφαρμογής τα τρωτά σημεία SQL Injection που έχετε εκμεταλλευτεί στις προηγούμενες δραστηριότητες. Για το σκοπό αυτό θα πρέπει επεξεργαστείτε τον **κώδικα php** στα αρχεία:

- **unsafe_home.php**, και
- **unsafe_edit_backend.php**

που βρίσκονται στον κατάλογο **/var/www/SQLInjection**.

Αφού κάνετε τη διόρθωση, θα πρέπει να δοκιμάσετε κάποιες από τις προηγούμενες επιθέσεις ώστε να ελέγξετε αν μπορείτε ακόμα να εκμεταλλευτείτε την ευπάθεια ή όχι.

Στην αναφορά σας θα πρέπει να εξηγήσετε τον τρόπο (τις αλλαγές που κάνατε στον κώδικα) με τον οποίο διορθώσατε τα συγκεκριμένα αρχεία. Επίσης, θα πρέπει να συμπεριλάβετε screenshots από την εκτέλεση των επιθέσεων μετά την διόρθωση, ώστε να αποδειχθεί η επιτυχία της διόρθωσης.

Υποβολή

Θα πρέπει να υποβάλετε μια λεπτομερή **εργαστηριακή αναφορά** (σε PDF) που θα καλύπτει όλες τις ενέργειες που κάνατε σε κάθε δραστηριότητα, τα αποτελέσματα και τις παρατηρήσεις σας. Στην περιγραφή σας χρησιμοποιείτε στιγμιότυπα οθόνης και αποσπάσματα κώδικα, σύμφωνα με τις οδηγίες κάθε δραστηριότητας.